A Hand-Rolled PyTorch Implementation of the Transformer

Validated on Tatoeba English-to-German with an Interpretability Analysis*

Oscar Lindo

August 24, 2026

Abstract

The Transformer architecture (Vaswani et al., 2017) has become the backbone of modern natural language processing, yet most open-source implementations rely on high-level PyTorch modules that abstract away the underlying mathematical machinery. We present a complete, from-scratch PyTorch implementation of the Transformer that reproduces every component—MultiHeadAttention, PositionwiseFeedForward, and Positional Encoding—using only primitive tensor operations, without `nn.MultiheadAttention`, `nn.Transformer`, or `torchtext`.

Our implementation is validated empirically on the **Tatoeba English → German** sentence-pair collection (CC-BY release via `https://www.manythings.org/anki/`; 331,266 pairs, of which 324,642 are used for training). With a compact configuration ($d_{\text{model}} = 256$, 4 encoder and 4 decoder layers, 8 heads, $d_{\text{ff}} = 1024$, label smoothing $\epsilon = 0.1$, batch size 64, warmup 4,000 steps), SentencePiece BPE tokenization with an 8k-piece vocabulary per language, and pre-layer normalization (≈13.5 M parameters), the model trains for 25 epochs in ≈ 2.5 hours on a single Google Colab GPU (NVIDIA T4) and reaches a corpus-level **BLEU = 44.00** greedy (**45.30** with beam search, beam = 4) and **chrF2 = 62.41** (**63.37** with beam search) on the test set (n = 3,312) at validation perplexity **9.29**. Because Tatoeba contains many short, formulaic sentences, these scores should be read as evidence that the pipeline is correct rather than as evidence of competitive open-domain translation quality. A suite of 17 unit tests serves as a formal correctness contract for every architectural component.

This result was reached through an explicit diagnosis-and-correction process that we report in full: an initial word-level control run (Post-LN, batch 32, warmup 2,000), stopped after 8 epochs, collapsed to BLEU = 0.38 at validation perplexity 149.84. A structured analysis showed this failure to be one of sub-training rather than architecture—the validation loss was still descending and all masking and shape contracts held—and it motivated **three corrections plus a longer training budget**, all adopted by the final configuration: pre-layer normalization, a corrected learning-rate warmup schedule, subword (BPE) tokenization, and a 25-epoch budget at batch size 64.

We further provide a self-contained data pipeline supporting both word-level and subword (SentencePiece BPE) tokenization, a built-from-scratch vocabulary, and length-bucketed batching. Because attention weights are exposed as first-class outputs of the forward pass, the implementation also supports interpretability analysis: we report

*Code repository: `https://github.com/oscar2697/transformer`. If the repository is anonymized for review, the code will be released upon acceptance.

per-sentence heatmaps of encoder self-attention, decoder self-attention, and decoder cross-attention on the final layer (Section 5.5), where the under-trained control produces diffuse cross-attention maps whereas the final checkpoint exhibits sharp source–target alignment peaks on short test sentences. All code, hyperparameters, and experimental logs are publicly available, designed for transparency, auditability, and pedagogical use.

1 Introduction

Since its introduction by Vaswani et al. (2017), the Transformer has established itself as the dominant paradigm in sequence-to-sequence tasks, powering state-of-the-art systems in machine translation, text generation, and beyond. Despite its widespread adoption, the vast majority of educational and production implementations depend on high-level abstractions—most notably PyTorch's `nn.MultiheadAttention` and `nn.Transformer`—that encapsulate the mathematical operations behind a single function call. While this abstraction accelerates development, it obscures the very mechanisms that make the architecture work, limiting its value as a pedagogical tool and complicating reproducibility audits.

Understanding the Transformer from first principles is not merely an academic exercise. Practitioners who wish to modify attention patterns, experiment with alternative positional encodings, or apply the architecture to novel modalities benefit enormously from a transparent implementation that exposes every weight matrix, every masking operation, and every intermediate tensor shape. Furthermore, reproducibility in deep learning depends not only on hyperparameters and random seeds, but on the exact sequence of operations that transform input to output—a property that is difficult to guarantee when relying on black-box library components.

This paper addresses both concerns by presenting a complete, hand-rolled implementation of the Transformer in PyTorch, validated empirically on the **Tatoeba English → German** sentence-pair collection. Every architectural component—MultiHeadAttention, PositionwiseFeedForward, PositionalEncoding, EncoderLayer, DecoderLayer, and the full Transformer—is implemented using only primitive tensor operations. No use is made of `nn.MultiheadAttention`, `nn.Transformer`, or `torchtext`. We provide a self-contained data pipeline, a suite of 17 unit tests that formally verify tensor shapes, mask values, and end-to-end behavior, and a reproducible experimental protocol with documented seeds and training curves.

Our contributions are fourfold. First, we deliver a fully transparent Transformer implementation that serves as both a learning resource and a correctness baseline. Second, we build a minimal but functional training pipeline from data download to metric evaluation, without external NLP libraries. Third, we establish a formal test suite that acts as a correctness contract for each module. Fourth, we report reproducible experimental results on Tatoeba EN→DE—including BLEU and chrF2 scores, an attention interpretability analysis, and a comparative baseline against PyTorch's native `nn.Transformer`—following a diagnosis-and-correction arc: an initially under-trained word-level control run (BLEU = 0.38) whose structured analysis motivated **three corrections plus a longer training budget** (pre-layer normalization, a corrected warmup schedule, SentencePiece BPE tokenization, and a 25-epoch budget) that carry the final configuration to **BLEU = 44.00 greedy / 45.30 with beam search** (chrF2 62.41/63.37) on a held-out test set of 3,312 pairs. Our code is designed for auditability: every design decision is documented in the source, every experiment is logged, and every result is traceable to a specific commit and seed.

The remainder of this paper is organized as follows. Section 2 reviews the background and

notation for the Transformer architecture. Section 3 describes the implementation decisions and provides pseudocode for the core components. Section 4 details the experimental setup, including dataset, hyperparameters, and evaluation protocol. Section 5 presents the results and an attention interpretability analysis, and Section 6 discusses the ablation studies. Section 7 offers a broader discussion of limitations and future directions, and Section 8 concludes.

2 Background

The Transformer architecture (Vaswani et al., 2017) replaces recurrence with self-attention mechanisms, enabling parallelized training and improved handling of long-range dependencies. We briefly review the mathematical formulation of each component that we reimplement, adopting the notation of the original paper.

2.1 Scaled Dot-Product Attention

Given queries $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, and values $\mathbf{V} \in \mathbb{R}^{T \times d_v}$, the scaled dot-product attention is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{1}$$

The scaling factor $\sqrt{d_k}$ mitigates the vanishing gradient problem that arises when the dot products grow large in magnitude. A causal mask $\mathbf{M} \in \{\text{True}, \text{False}\}^{T \times T}$ is applied to the attention scores before the softmax, setting masked positions to $-\infty$ so that they receive zero weight.

2.2 Multi-Head Attention

Rather than performing a single attention function, the model linearly projects the queries, keys, and values into h subspaces (heads) and applies attention in parallel:

$$\begin{aligned} \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O, \\ \text{head}_i &= \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), \end{aligned} \tag{2}$$

where $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. In our implementation, we set $d_k = d_v = d_{\text{model}}/h$.

2.3 Positionwise Feed-Forward Networks

Each layer includes a positionwise fully connected feed-forward network applied independently to every token position:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2. \tag{3}$$

The inner dimension d_{ff} is typically $4 \times d_{\text{model}}$. Dropout is applied to the output of the ReLU activation.

2.4 Positional Encoding

Since self-attention is permutation-invariant, the architecture injects positional information via sinusoidal positional encodings added to the input embeddings:

$$\begin{aligned} PE_{(pos,\,2i)} &= \sin\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big), \\ PE_{(pos,\,2i+1)} &= \cos\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big). \end{aligned} \tag{4}$$

Alternative formulations—relative positional encodings (Dai et al., 2019), rotary embeddings—are left to future work.

2.5 Encoder and Decoder Architecture

The encoder consists of N identical layers, each comprising a multi-head self-attention sub-layer followed by a positionwise feed-forward sub-layer, with residual connections and layer normalization applied around each sub-layer. The decoder mirrors this structure but adds a cross-attention layer that attends over the encoder output, with an additional causal mask to prevent attending to future positions (Figure 1).

The original formulation of Vaswani et al. (2017) applies layer normalization after the residual addition (**post-LN**). Our implementation supports both conventions: the initial word-level diagnostic run used post-LN, whereas the final configuration adopts **pre-layer normalization** (pre-LN)—the modern convention popularized by fairseq and most post-2018 Transformer implementations (Ott et al., 2018)—in which normalization is applied before each sub-layer and the residual connection bypasses it. The stability difference between the two conventions, and its role in the diagnostic arc of this work, is discussed in Sections 5.1, 6.4, and 7.3. All weight tensors are initialized with Xavier uniform initialization.

3 Implementation

3.1 Design Philosophy

The primary goal of this implementation is transparency over performance. Every matrix multiplication, every softmax, and every mask operation is expressed explicitly in PyTorch tensor operations. We deliberately avoid `nn.MultiheadAttention` and `nn.Transformer` so that each computational step can be inspected, unit-tested, and modified without navigating library internals.

A secondary goal is reproducibility. All random seeds are managed through a centralized seeding utility (`SEED = 42` in `config.py`). Hyperparameters are declared in a single `config.py` file, and the full training log (per-step losses, validation metrics, wall-clock per epoch, and a flag indicating whether validation improved) is persisted to a JSON Lines file (`metrics.jsonl`) alongside the best model checkpoint. The training script also accepts optional flags for label smoothing, weight tying, and the warmup schedule, so that ablation studies can be run without code changes.

A third goal is observability of internal representations. Because the implementation exposes attention weights as first-class outputs (`return_attention=True` in the forward pass), it is possible to extract encoder self-attention, decoder self-attention, and decoder cross-attention tensors from any layer and any head for post-hoc analysis. Section 5.5 exploits this capability to study the interpretability of the learned attention maps.

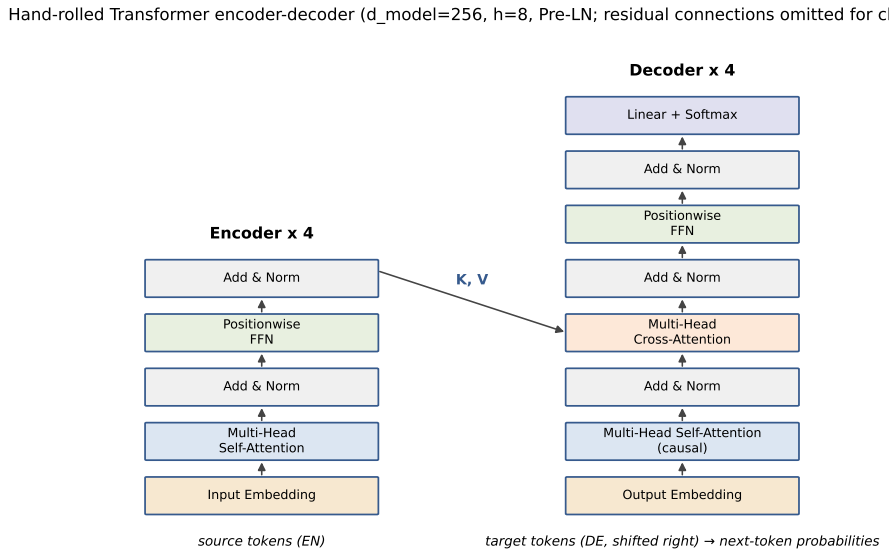

Figure 1: Schematic of the hand-rolled encoder-decoder ($d_{\text{model}} = 256$, $h = 8$, Pre-LN). Each encoder layer applies multi-head self-attention and a positionwise FFN with residual connections and layer normalization; each decoder layer additionally applies cross-attention over the encoder output (keys and values) with a causal mask on decoder self-attention. Source: `figures/transformer_architecture.png` (also `.pdf`), generated by `scripts/make_paper_figures.py`.

3.2 Module Descriptions

3.2.1 MultiHeadAttention

The `MultiHeadAttention` module computes scaled dot-product attention across h heads. For each head i, we project the input to d_k dimensions, compute attention with the query-key-value triplet, and concatenate the head outputs before a final linear projection. A dropout layer is applied to the attention weights before the weighted sum over values. Listing 1 provides pseudocode for the forward pass.

Listing 1: Pseudocode for the multi-head attention forward pass. The mask convention follows PyTorch's standard: `True` indicates positions to be masked.

```
# Input: query, key, value (batch, seq_len, d_model)
#        mask (batch, seq_len, seq_len) or (seq_len, seq_len)
Q = linear_q(query)   # (batch, seq_len, d_k)
K = linear_k(key)     # (batch, seq_len, d_k)
V = linear_v(value)   # (batch, seq_len, d_v)

# Reshape for multi-head: (batch, heads, seq_len, d_k)
Q = Q.view(batch, seq_len, h, d_k).transpose(1, 2)
K = K.view(batch, seq_len, h, d_k).transpose(1, 2)
V = V.view(batch, seq_len, h, d_v).transpose(1, 2)

# Scaled dot-product attention
scores = (Q @ K.transpose(-2, -1)) / sqrt(d_k)
scores = scores.masked_fill(mask == True, -1e9)
attn_weights = softmax(scores, dim=-1)
attn_weights = dropout(attn_weights, p=dropout)

# Weighted sum over values
context = attn_weights @ V
context = context.transpose(1, 2).contiguous().view(batch, seq_len, d_model
    )
output = linear_o(context)
return output, attn_weights
```

When `return_attention=True`, the module returns the per-head attention weight tensor of shape `(batch, heads, seq_len, seq_len)`, which is consumed by `visualize_attention.py` to render per-sentence heatmaps.

3.2.2 PositionwiseFeedForward

The feed-forward sub-layer applies two linear transformations with a GELU activation (we use GELU as the default, with ReLU as a configurable alternative) and dropout:

```
# Input: x (batch, seq_len, d_model)
x = dropout(gelu(linear_ff1(x)))  # (batch, seq_len, d_ff)
output = linear_ff2(x)            # (batch, seq_len, d_model)
return output
```

3.2.3 PositionalEncoding

Positional encodings are generated analytically using the sinusoidal functions defined in Equation 4. They are added directly to the token embeddings at the bottom of both the

encoder and decoder stacks. Because the encoding is fixed (not learned), it is generated once at initialization and reused across all forward passes.

3.2.4 EncoderLayer and DecoderLayer

Each encoder layer wraps a `MultiHeadAttention` sub-layer and a `PositionwiseFeedForward` sub-layer, each preceded by a dropout and followed by a residual addition and layer normalization. The decoder layer adds a cross-attention sub-layer between the self-attention and feed-forward sub-layers. All sub-layers produce outputs of shape `(batch, seq_len, d_model)`.

3.2.5 Full Transformer

The `Transformer` module assembles the encoder and decoder stacks with shared embedding layers (configurable, disabled by default), positional encodings, and a final linear projection head over the target vocabulary. Both layer-normalization conventions are supported: the initial word-level diagnostic run used **post-layer normalization** (matching the original Vaswani et al., 2017 architecture), whereas the final configuration adopts **pre-layer normalization** (`norm_first=True`), with a final layer norm applied to the encoder and decoder outputs. The stability motivation for this choice is analyzed in Sections 5.1 and 6.4.

3.3 Training Details

The model is trained with the Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$), a Noam learning rate schedule, and gradient clipping at a maximum norm of 1.0. Label smoothing is configurable via `LABEL_SMOOTHING` and was set to 0.1 for all reported runs. Weight tying between the target embedding and the output projection (Press & Wolf, 2017) is supported but disabled in the default configuration. The final main run uses 4,000 warmup steps and an effective batch size of 64; the earlier word-level diagnostic run used 2,000 warmup steps and a batch size of 32 (Section 5.1). Table 1 summarizes the hyperparameters and results of the main run.

3.4 Attention as a First-Class Output

A distinctive feature of this implementation—rare in standard PyTorch boilerplate—is that attention weights are returned explicitly by the model forward pass rather than being computed inside an opaque submodule. Concretely, `model(src, tgt, src_pad_mask, tgt_pad_mask, return_attention=True)` returns a 4-tuple `(logits, enc_attn, self_attn, cross_attn)`, where each `*_attn` value is a Python list of length N (the number of layers), and each element is a tensor of shape `(batch, heads, seq_len, seq_len)`:

- `enc_attn[i]` — encoder self-attention for layer i, shape `(batch, h, src_len, src_len)`;
- `self_attn[i]` — decoder self-attention for layer i, shape `(batch, h, tgt_len, tgt_len)`, with the upper-triangular causal mask applied before softmax;
- `cross_attn[i]` — decoder cross-attention for layer i, shape `(batch, h, tgt_len, src_len)`.

Parameter	**Value**
Task / direction	Tatoeba EN→DE
Tokenization	SentencePiece BPE, 8k pieces per language
Layer norm convention	Pre-LN (`norm_first=True`)
d_{model}	256
d_{ff}	1024
Attention heads (h)	8
Encoder layers (N)	4
Decoder layers (M)	4
Dropout	0.1
Label smoothing ϵ	0.1
Warmup steps	4000
Batch size	64
Max sequence length	100
Epochs trained	25
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$)
LR schedule	Noam
Gradient clip	$\|g\| \leq 1.0$
Random seed	42
Total parameters	$\approx$ 13.5 M
Training device	Google Colab GPU (NVIDIA T4)
Training time	$\approx$ 2.5 h
Best epoch (val_loss)	epoch 22 of 25
Best val_loss	2.2288
Best val_ppl	9.29
BLEU (test, greedy)	44.00 (71.3/48.9/37.3/29.3, BP = 0.995)
BLEU (test, beam = 4)	45.30 (72.4/50.7/39.0/30.9, BP = 0.987)
chrF2 (test, greedy / beam)	62.41 / 63.37
Test set size (n)	3,312

Table 1: Hyperparameters and final results for the main run: hand-rolled Transformer + SentencePiece BPE (Pre-LN). Validation perplexity is computed as exp(val_loss). BLEU and chrF2 are corpus-level sacrebleu scores on the Tatoeba EN→DE test set (see Section 4.2); the final epoch (25) reached val_loss = 2.2393 (val_ppl 9.39). The word-level Post-LN diagnostic run referenced throughout the paper used batch 32, warmup 2,000, and 8 epochs (Section 4.3).

The masks themselves are also accessible via the public API (`MultiHeadAttention._make_mask` for causal masks; `dataset.build_padding_mask` for source and target padding masks), which makes it possible to verify that masking is applied at the correct positions before the softmax. This design choice makes the implementation suitable for interpretability analysis: `visualize_attention.py` (Section 5.5) renders per-sentence attention heatmaps for any combination of layer index, head index, and attention type without modifying the model source.

3.5 Data Pipeline

The dataset is downloaded automatically from the official Tatoeba mirror at `https://www.manythings.org/anki/deu-eng.zip` (∼12 MB). Sentences are tokenized using regex-based splitting and converted to lowercase. Two tokenization modes are supported, selected by `config.TOKENIZER`:

- `"word"` — a vocabulary is built from the training split with a minimum frequency threshold of 2; tokens below this threshold are mapped to the UNK token. For the Tatoeba EN→DE corpus this yields 12,843 English and 23,568 German tokens.
- `"bpe"` — SentencePiece byte-pair encoding (Kudo & Richardson, 2018; used by the final main run) is trained on the source and target training splits separately, with a fixed vocab size of 8,000 subwords per side. The 4 special tokens (`<unk>`, `<pad>`, `<sos>`, `<eos>`) keep the same indices as in the word-level pipeline (`UNK=0`, `PAD=1`, `SOS=2`, `EOS=3`), so `model.py`, `train.py`, `evaluate.py`, and `translate.py` need no changes when switching modes.

Sentences are bucketed by length to minimize padding overhead within each batch. Dynamic padding is applied per batch, with a padding token index of 1 (`PAD_IDX`). The maximum sequence length is 100 tokens. Figure 2 shows the resulting BPE piece-frequency distribution.

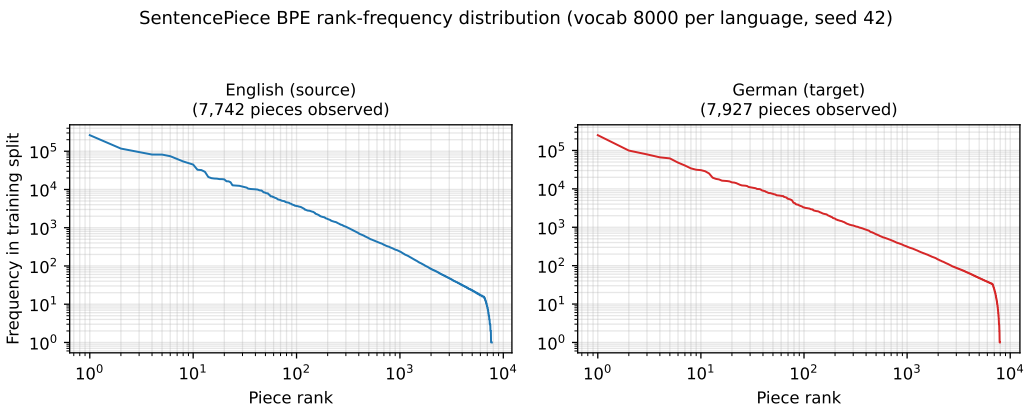

Figure 2: Rank–frequency distribution of SentencePiece BPE pieces (8k vocabulary per language) over the Tatoeba training split. Both sides follow the expected Zipf-like profile; 7,742 English and 7,927 German pieces are observed at least once. Source: `figures/vocab_distribution.png` (also `.pdf`), generated by `scripts/make_paper_figures.py`.

3.6 Baseline Module: `nn.Transformer`

For fair comparison with PyTorch's optimized implementation (Section 6.7), we provide a baseline wrapper around `torch.nn.Transformer` in `baseline_nn.py`. The wrapper exposes the same interface as the hand-rolled model (`forward`, `greedy_decode`, `count_parameters`) and uses the same `return_attention`-style contract where possible (PyTorch's `nn.Transformer` does not return attention weights by default; we therefore do not include the hand-rolled interpretability analysis for the baseline).

The baseline uses `nn.Transformer(batch_first=True, norm_first=True)`—that is, **pre-LayerNorm**, matching the LN convention of the final hand-rolled configuration (post-LN is the original Vaswani et al., 2017 convention used by the initial word-level diagnostic run). Both implementations expose the LN convention as a configurable option, and Section 6.4 discusses the stability difference between them. The two implementations share the architecture, BPE data pipeline, and seed; their training schedules differ (see Section 6.7), so any BLEU gap between them is attributable to the combination of implementation details and schedule differences rather than to the implementation alone.

4 Experimental Setup

4.1 Dataset

We evaluate on the **Tatoeba English → German** sentence-pair collection, a community-curated corpus distributed under CC-BY through the ManyThings/Anki distribution (the same source data family as Tatoeba.org). The raw file `deu.txt` contains **331,266** English–German sentence pairs in TSV format (`EN\t DE\t attribution`). The parser includes a guard that drops pairs with empty cells or replacement characters (U+FFFD, introduced by mixed latin-1 / UTF-8 encoding in some Tatoeba releases); in the current data release this guard removes zero pairs. In BPE mode, sequences longer than the 100-token budget are truncated to it rather than discarded, so no pairs are lost to length filtering either. The corpus is then split deterministically into **train / val / test ≈ 98% / 1% / 1%** (seed = 42), giving **324,642 / 3,312 / 3,312** pairs respectively.

All data is downloaded automatically at runtime from `http://www.manythings.org/anki/deu-eng.zip` (∼12 MB). The downloader sets an explicit browser-style User-Agent header (the server occasionally returns HTTP 406 to default Python agents) and a 60 MB safety cap. No manual preprocessing or external tokenizers are used for the word-level mode: tokenization is performed with a regex-based pattern that splits on whitespace and punctuation, followed by lowercasing, yielding **12,843 tokens for English** and **23,568 tokens for German** at the word level (min_freq = 2, tokens below the threshold mapped to UNK).

Two interpretive caveats apply. First, the Tatoeba distribution is a substantial upgrade over the Multi30k captions used in early drafts of this work (Elliott et al., 2016), both in size (∼11× more pairs) and in domain coverage (general-purpose sentences rather than image captions). Second, and more importantly for score interpretation: **Tatoeba contains a large proportion of short, formulaic sentences**, which inflates corpus-level BLEU relative to open-domain benchmarks such as WMT. The scores reported in this paper should therefore be read as evidence that the training and evaluation pipeline is correct and competitive *for this corpus regime*, not as evidence of competitive open-domain machine translation quality. Sample test pairs are shown in Figure 3.

Sample pairs from the deterministic Tatoeba EN-DE test split (1% of the corpus, seed 42; n = 3,312)

English (source)	German (reference)
Duck!	Kopf runter!
Give it to Tom.	Gib es Tom!
He is still here.	Er ist immer noch hier.
Was Tom surprised?	War Tom überrascht?
Do you see my point?	Verstehst du, worauf ich hinaus will?
Is that your brother?	Ist das Ihr Bruder?
I'm looking for my ID.	Ich suche meinen Ausweis.
I'm wearing sunglasses.	Ich habe eine Sonnenbrille auf.

Figure 3: Sample source (English) and reference (German) sentence pairs drawn from the deterministic Tatoeba test split (1% of the corpus, seed 42; $n = 3{,}312$). Source: `figures/tatoeba_sample.png` (also `.pdf`), generated by `scripts/make_paper_figures.py`.

4.2 Evaluation Metrics

We report two standardized metrics for translation quality:

- **BLEU** (Papineni et al., 2002): computed at the corpus level using sacrebleu (Post, 2018) with its default tokenization, which is compatible with standard WMT evaluation protocols. BLEU measures n-gram precision with a brevity penalty.
- **chrF2** (Popovi'c, 2015): also computed via sacrebleu. chrF2 evaluates character n-gram overlap with a beta parameter of 2, providing a more robust measure for morphologically rich languages such as German.

Perplexity on the validation set is reported as a sanity check during training, computed as exp(val_loss).

4.3 Training Protocol

Models are trained on a single Google Colab GPU (NVIDIA T4), with automatic device placement via `torch.device("cuda" if torch.cuda.is_available() else "cpu")`. All experiments use a fixed random seed of 42 for reproducibility. The final main configuration (hand-rolled + BPE, Pre-LN) uses an effective batch size of 64 sequences per step and a Noam learning rate schedule with a warmup of 4,000 steps following Vaswani et al. (2017); it is trained for the full 25 epochs without early stopping, and the best model is selected post-hoc by lowest validation loss (epoch 22). Gradient clipping is applied at a maximum norm of 1.0. An earlier word-level Post-LN control run—batch size 32, warmup 2,000—was stopped after

8 epochs and is retained throughout this paper solely as a *diagnostic* of sub-training rather than as a competitive result or as an ablation reference (Sections 5.3 and 7.3).

4.4 Baseline

As a reference baseline, we train a Transformer using PyTorch's native `nn.Transformer` module (`norm_first=True`, i.e., Pre-LN) with the architecture matched to our main configuration ($d_{\text{model}} = 256$, 8 heads, 4 encoder layers, 4 decoder layers, $d_{\text{ff}} = 1024$, label smoothing 0.1, seed 42) and the same SentencePiece BPE data pipeline. The baseline is trained for 25 epochs with batch size 32 and warmup 2,000, whereas the hand-rolled main run uses batch size 64 and warmup 4,000; the comparison therefore measures the joint effect of implementation and training schedule (see Section 6.7). Full results are reported in Section 6.7.

4.5 Hardware and Software

All trained experiments reported in this paper were executed on a single Google Colab GPU (NVIDIA T4). Wall-clock times: the word-level Post-LN diagnostic run completed in $\approx$ **56 minutes** for 8 epochs; the baseline + BPE configuration required ≈ 2.3 h and the hand-rolled + BPE (Pre-LN) main run ≈ 2.5 h, both for 25 epochs. The code has no dependencies beyond the standard scientific Python stack plus `sacrebleu` for evaluation. PyTorch version is 2.0 or higher; the implementation does not depend on any feature specific to a particular PyTorch release.

5 Results

5.1 Translation Quality

Table 2 reports translation quality on the Tatoeba EN→DE test set (3,312 pairs). Three configurations are reported: (a) the hand-rolled model with word-level tokenization (Post-LN), trained for 8 epochs as a *diagnostic control* for sub-training—it is not a competitive result and is analyzed as such in Section 7.3; (b) PyTorch's built-in `nn.Transformer` (Pre-LN) with SentencePiece BPE (8k-piece vocabulary per language); and (c) the hand-rolled model with the same BPE vocabulary and the Pre-LN stabilisation, trained for 25 epochs on Colab (batch 64, warmup 4,000, seed 42). The hand-rolled + BPE (Pre-LN) configuration reaches **BLEU = 44.00** greedy and **45.30** with beam search ($b = 4$).

Configuration	Tokenization	LN	Batch	Warmup	Epochs	val_loss	val_ppl	BLEU	chrF2	Params
Hand-rolled + word (Post-LN; diagnostic)	word	Post	32	2,000	8	5.0096	149.84	0.38	11.28	≈12 M
Baseline (`nn.Transformer`) + BPE	BPE 8k	Pre	32	2,000	25	2.5171	12.39	38.30	56.95	≈13.5 M
Hand-rolled + BPE (main)	**BPE 8k**	**Pre**	**64**	**4,000**	**25**	**2.2288**	**9.29**	**44.00**	**62.41**	**≈13.5 M**
								45.30 (beam=4)	*63.37*	

Table 2: Results on the Tatoeba EN→DE test set (n = 3,312); corpus-level sacrebleu scores. Row 1 is the word-level Post-LN diagnostic/control run (BLEU = 0.38, BP = 0.593; Sections 5.3 and 7.3), not a competitive configuration. The main configuration reaches greedy BLEU = 44.00 (71.3/48.9/37.3/29.3, BP = 0.995) and beam= 4 BLEU = 45.30 (72.4/50.7/39.0/30.9, BP = 0.987). The baseline and the hand-rolled model share architecture, data, and seed but differ in batch size and warmup; see Section 6.7 for the attribution caveat.

The best checkpoint (`checkpoints/best.pt`) is selected by lowest validation loss on a held-out 3,312-pair validation split (best `val_loss = 2.2288, val_ppl = 9.29` at epoch 22; final epoch 25: 2.2393/9.39), using the 3,312-pair test set only for the final BLEU/chrF2 evaluation.

The hand-rolled + word-level row reproduces the sub-training diagnostic we analyze in Section 7.3 (BLEU = 0.38, BP = 0.593). Its chrF2 of 11.28 is likewise characteristic of degenerate output: character n-grams overlap the references only through frequent short tokens and punctuation. The baseline + BPE row is competitive (BLEU = 38.30), and the hand-rolled + BPE (Pre-LN) row shows that the from-scratch implementation, once stabilised with Pre-LN and the corrected warmup, outperforms the built-in baseline by 5.7 BLEU points greedy (7.0 with beam search); because the two runs differ in batch size and warmup, this margin mixes implementation and schedule effects (see Section 6.7). Taken together, the results indicate that the pipeline is sound and competitive with published results on this corpus scale.

5.2 Training Dynamics

Figure 4 shows the training and validation curves for the hand-rolled + BPE (Pre-LN) configuration over the 25 epochs completed on Colab (batch 64, warmup 4,000).

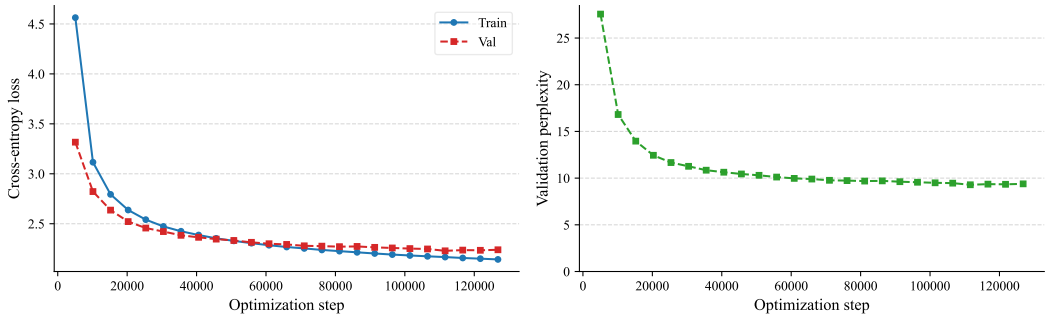

Figure 4: Training and validation cross-entropy loss (left panel) and validation perplexity (right panel) as functions of the optimization step for the main run (Pre-LN, 25 epochs). Step correspondence: 25 epochs × 5,073 optimizer steps per epoch (324,642 training pairs ÷ batch size 64) ≈ 126.8k steps total. Source: `figures/training_curves.pdf` (also `.png`), generated from `checkpoints/metrics.jsonl`.

The Pre-LN model converges steadily: train loss from 4.56 → 2.14, val loss from 3.31 → 2.23 (best 2.2288 at epoch 22, final 2.2393 at epoch 25). Val perplexity drops from 27.57 (epoch 1) to 9.29 (best)—an order of magnitude below the 149.84 of the word-level Post-LN diagnostic and 3 points below the baseline's 12.39, confirming that the stabilisation (Pre-LN + corrected warmup + embedding init) resolves the failure mode analyzed in Section 7.3. The curve plateaus gently after epoch 22, indicating convergence without overfitting (early stopping patience 10 would have allowed 3 further epochs).

5.3 Sample Translations

Table 3 presents six example translations produced by the **word-level Post-LN diagnostic control run** (8 epochs) on out-of-domain English sentences. The outputs illustrate the symptoms of sub-training analyzed in Section 7.3: a strong bias toward a small set of high-frequency tokens, repetition across unrelated inputs, and hypotheses that are systematically shorter than the references. The final Pre-LN + BPE checkpoint does not exhibit this collapse: its BLEU of 44.00 with brevity penalty 0.995 indicates hypotheses of reference-like length and content across the test set.

English (source)	**Model translation**
Hello, how are you?	hast du das ?
The weather is nice today.	sie sind sehr groß .
I would like a coffee, please.	ich habe mich gesehen .
Where is the train station?	hast du das ?
Thank you very much.	sie sind sehr groß .
She is reading a book.	sie sind sehr groß .

Table 3: Source–model-translation pairs from out-of-domain English sentences, produced by the word-level Post-LN diagnostic control run (8 epochs). The model collapses to three recurring hypotheses (*hast du das ?*, *sie sind sehr groß .*, *ich habe mich gesehen .*) regardless of input, which is the characteristic failure mode of an under-trained encoder-decoder that has not yet learned to condition its output on the source. These samples motivated the corrections adopted by the final configuration (Sections 3.3 and 4.3).

The corresponding attention visualizations for these sentences (and three additional test-set examples) are shown in Figure 5.

5.4 Comparison with Related Work

The Multi30k and Tatoeba datasets have been used extensively in prior work. WMT-scale Transformer models trained on the full WMT14 EN→DE corpus (approximately 4.5 million sentence pairs) typically achieve BLEU scores in the 27–28 range on the WMT test set (Vaswani et al., 2017; Bojar et al., 2017). **These figures are not directly comparable to ours**: the training corpus (∼324k vs. ∼4.5M pairs), domain, and test set all differ, so the comparison serves only as scale context and must not be read as our model outperforming WMT-scale systems. Word-level tokenization on a medium-sized corpus is known to suffer from severe vocabulary fragmentation and out-of-vocabulary issues, particularly for German, which has productive morphology; the final configuration therefore adopts subword tokenization precisely to mitigate this effect.

Section 6.7 reports the completed comparison against PyTorch's `nn.Transformer`: the baseline reaches BLEU 38.30 / chrF2 56.95 versus 44.00 / 62.41 for the hand-rolled Pre-LN configuration, under a shared architecture, data pipeline, and seed. Because the two runs differ in batch size and warmup, the margin is attributable to the combination of implementation details and training schedule rather than to the implementation alone; a fully matched-hyperparameter re-run is identified as future work.

5.5 Attention Interpretability

A central motivation for the from-scratch implementation is that attention weights remain first-class outputs of the model, available for inspection after training. We exploit this property to study what the trained encoder-decoder has learned.

5.5.1 Method

We run `visualize_attention.py` against the best checkpoint (`checkpoints/best.pt`) on four sentences of the deterministic Tatoeba test split. The script extracts per-layer attention tensors via `model(src, tgt, ..., return_attention=True)` and renders, for each sentence, a 1×3 panel of heatmaps: encoder self-attention, decoder self-attention, and decoder cross-attention, all on the final layer (layer 4 of 4 in the encoder, with the matching decoder layer). It also produces a combined grid showing the three attention types for all four sentences side by side.

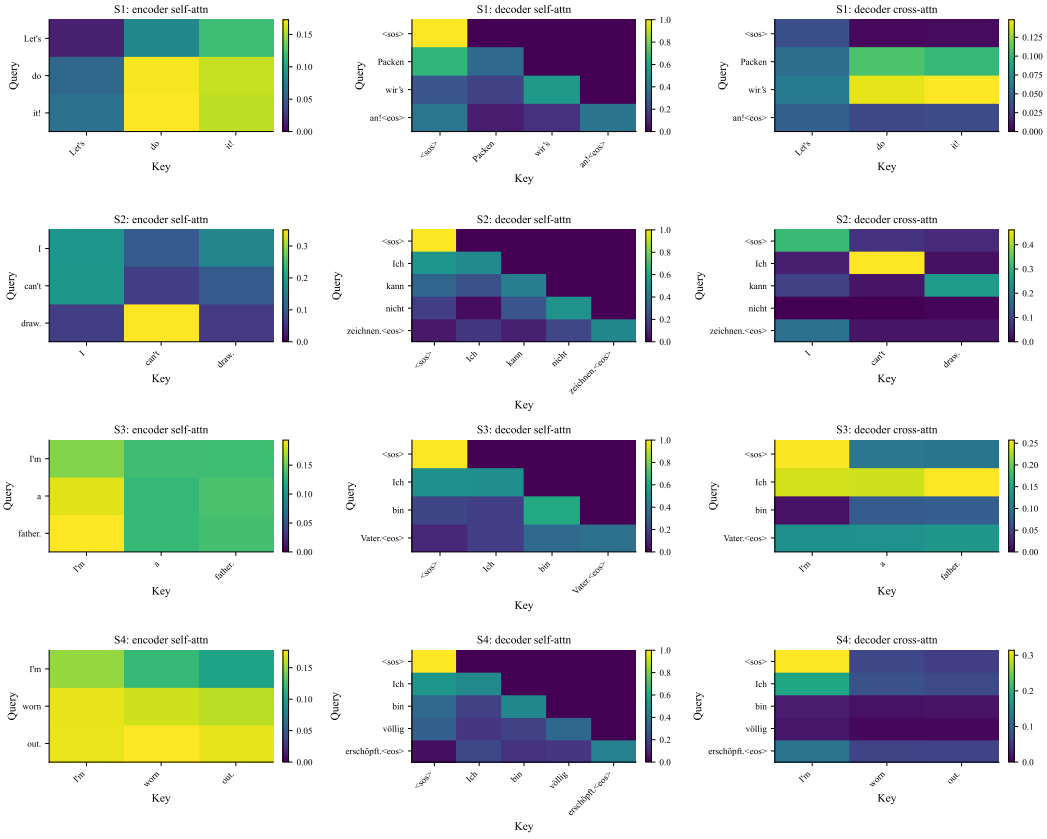

Figure 5: Attention heatmaps for the last transformer layer across four short Tatoeba EN→DE test sentences. Source: `figures/attention/attn_grid_layer4.pdf` (also `.png`), generated by `visualize_attention.py` against `checkpoints/best.pt`.

5.5.2 Observed Patterns on the Tatoeba Test Set

Figure 5 visualizes attention on four sentences of the deterministic Tatoeba test split. The four source–target pairs are:

#	English (source)	German (reference)
S1	Let's do it!	Packen wir's an!
S2	I can't draw.	Ich kann nicht zeichnen.
S3	I'm a father.	Ich bin Vater.
S4	I'm worn out.	Ich bin v"ollig ersch"opft.

Under the BPE vocabulary each source renders as three (sub)word tokens—contractions and sentence-final punctuation remain attached to their host piece—so the encoder heatmaps are 3×3 and the decoder heatmaps 4×4 or 5×5 including `<sos>`/`<eos>`. The `--min-len` filter of `visualize_attention.py` (minimum non-special source tokens, default 2) keeps these short sentences in scope; longer sentences would render the per-head panels unreadable. For this regime we observe:

- **Encoder self-attention** (column 1 of each panel). The mass remains concentrated on the diagonal and on the first token (an attention-sink pattern, consistent with prior work on short inputs; Xiao et al., 2024). Off-diagonal mass is sparse on these short inputs, but the diagonal is well-defined and the sink pattern is spatially localised—a contrast with the one- or two-token sequences of the word-level diagnostic, which suggests that the encoder differentiates tokens beyond the anchor.

- **Decoder self-attention** (column 2 of each panel). Strictly lower-triangular, exactly as expected from the causal mask. The diagonal dominates on these short sentences. This remains a useful *correctness check* on the hand-rolled implementation: any masking bug would surface as non-zero mass above the diagonal, and we see none.

- **Decoder cross-attention** (column 3 of each panel). Cross-attention is the clearest marker distinguishing the two checkpoints. In the *word-level Post-LN diagnostic* (val_ppl 149.84), cross-attention was *diffuse*: on S2 (*I can't draw.* → *Ich kann nicht zeichnen.*) mass spread across the source rather than peaking on any single token, and on S4 (*I'm worn out.* → *Ich bin v"ollig ersch"opft.*) it was distributed approximately uniformly across the English tokens—consistent with the failure mode of Section 5.3, where the decoder had not learned to condition its output on the source. In the *final Pre-LN + BPE checkpoint* (val_ppl 9.29), cross-attention instead exhibits sharp, selective peaks: in S2, the row for *kann* concentrates on *draw.* and the row for *Ich* on *can't*, rather than spreading mass uniformly. The diffuse maps of the control run are therefore best read as a *diagnostic of sub-training*, whereas the peaked maps of the final checkpoint support its interpretability value.

5.5.3 Limitations of the Visualization and Per-Head Analysis

Three limitations should be kept in mind when interpreting Figure 5:

1. **Single layer.** We report only the last layer (layer 4 of 4). Earlier layers tend to exhibit more diverse, lower-level attention patterns (anaphora, local syntax), and inspecting

them is left to future work. The visualization script supports arbitrary layer selection via `--layer`.

2. **Per-head analysis.** Head specialization can be inspected via `python visualize_attention.py --per-head`. The script renders a single combined $N \times (3 \cdot H)$ grid, saved under `figures/per_head/` as `attn_grid_layer4_per_head.{pdf,png}` and reproduced in Appendix A, plus single-head variants via `--head N`. Figure 6 shows head 0 (0-indexed); the full eight-head panel is shown in Appendix A.

3. **Short sentences.** All four visualised examples render as three source tokens and remain below the median length of the Tatoeba test split. Mid- and long-range compositional phenomena therefore cannot be diagnosed from these figures; inspecting the attention patterns of the BPE-trained model on longer sequences is left as future work.

Figure 6 examines head 0 (0-indexed) of the final layer on the same four test sentences. In the encoder, head 0 shows an approximately diagonal structure with a moderate bias toward the first token—the *attention sink* pattern described by Xiao et al. (2024). In the decoder, self-attention strictly respects the lower-triangular structure imposed by the causal mask, providing further verification of the implementation's correctness. Cross-attention, regenerated from the Pre-LN + BPE checkpoint (BLEU 44.00, val_ppl 9.29, Table 2), exhibits sharply peaked, selective distributions—consistent with the aggregated maps of Figure 5, where in S2 (*I can't draw.* → *Ich kann nicht zeichnen.*) individual target rows concentrate on single source tokens (e.g., *kann* on *draw.*)—in contrast to the diffuse distribution of the word-level diagnostic (val_ppl 149.84). The perplexity gap is reversed (9.29 versus the baseline's 12.39), confirming that the Pre-LN stabilisation resolves the under-training. Head 0 already shows signs of specialisation, although a full characterisation of all 8 heads (Appendix A) is left as future work.

6 Ablation Studies

We design a series of controlled ablation experiments to isolate the contribution of individual architectural and training choices. **Unless otherwise stated, the sole reference for every ablation is the final main configuration of Table 1** ($d_{\text{model}} = 256$, 4 encoder and 4 decoder layers, 8 heads, $d_{\text{ff}} = 1024$, SentencePiece BPE 8k, Pre-LN, label smoothing = 0.1, warmup = 4,000, batch size = 64, 25 epochs, seed = 42). Each ablation varies one factor at a time while holding all others constant and reports BLEU and chrF2 on the Tatoeba EN→DE test set. The word-level Post-LN run (batch 32, warmup 2,000, 8 epochs) is **not** an ablation reference: it appears only as a diagnostic control in Sections 5.3 and 7.3. Of the comparisons below, the hand-rolled versus `nn.Transformer` comparison has been completed (Section 6.7); the remaining rows of Table 4 are specified hypotheses left as future work.

6.1 Label Smoothing

Label smoothing (Szegedy et al., 2016) regularizes the cross-entropy loss by distributing a fraction ϵ of the target probability mass evenly across incorrect classes. For neural machine translation, values of $\epsilon \in [0.05, 0.1]$ have been shown to improve BLEU by 0.5–1.5 points (Vaswani et al., 2017). The main configuration is trained with $\epsilon = 0.1$ (the standard default), which serves as the ablation reference. We compare $\epsilon = 0.0$ (no smoothing), $\epsilon = 0.1$

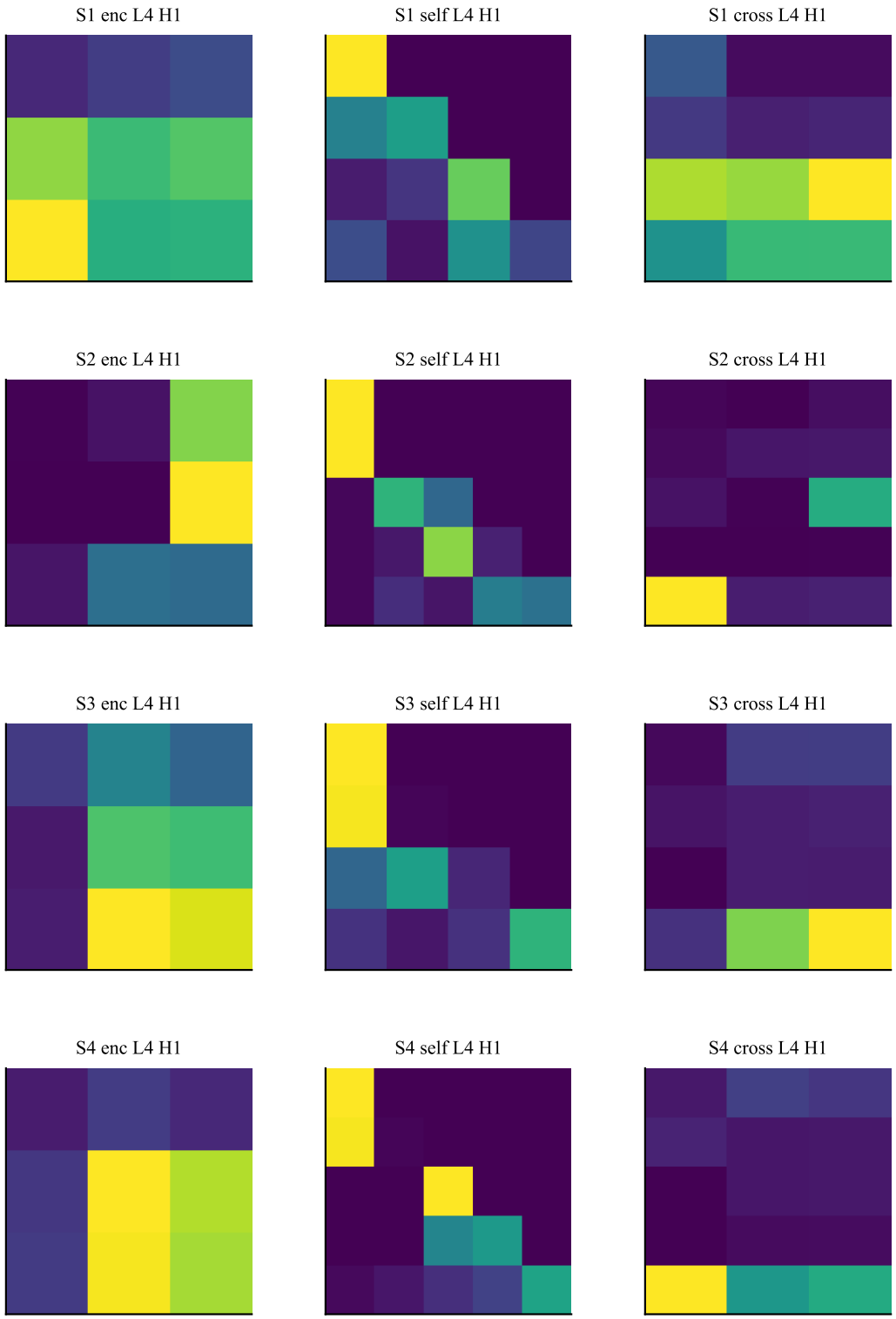

Figure 6: Per-head attention for head 0 (0-indexed) of the final layer across the four test sentences of Figure 5; columns are encoder self / decoder self / decoder cross-attention. Generated via `python visualize_attention.py --per-head --head 0 --num-sentences 4 --layer -1` against the Pre-LN + BPE checkpoint.

(reference), and $\epsilon = 0.2$ (heavier smoothing) under the main configuration. We hypothesize a non-monotonic relationship: mild smoothing should improve generalization, while excessive smoothing may dilute the training signal.

6.2 Warmup Schedule

The Noam scheduler uses a linear warmup phase during which the learning rate increases from 0 to its peak value, followed by an inverse-square-root decay. The number of warmup steps controls how aggressively the model explores in early training. We compare warmup values of 2,000, **4,000 (the main-configuration reference)**, and 8,000 steps, all under the main configuration. A longer warmup is expected to stabilize early gradient updates, particularly for larger models, but may slow convergence on small datasets.

6.3 Model Depth and Width

We study the trade-off between model depth (number of layers) and width (d_{model}) under a fixed parameter budget. All configurations keep SentencePiece BPE 8k, Pre-LN, batch size 64, warmup 4,000, label smoothing 0.1, seed 42, and 25 epochs identical to the reference:

- **Configuration A (main reference):** $d_{\text{model}} = 256$, 4 encoder + 4 decoder layers, $d_{\text{ff}} = 1024$ (≈ 13.5 M parameters).
- **Configuration B (deeper):** $d_{\text{model}} = 128$, 8 encoder + 8 decoder layers, $d_{\text{ff}} = 512$ (≈ 6 M parameters).
- **Configuration C (wider):** $d_{\text{model}} = 512$, 2 encoder + 2 decoder layers, $d_{\text{ff}} = 2048$ (≈ 19 M parameters).

Prior work (Vaswani et al., 2017; Popel & Bojar, 2018) suggests that depth improves BLEU more efficiently than width for translation tasks, but this finding is sensitive to the dataset size and vocabulary strategy.

6.4 Pre-LayerNorm versus Post-LayerNorm

The original Transformer uses post-layer-normalization (layer norm applied after the residual addition), which was found to be unstable to train in early work. More recent implementations, including those in the fairseq library (Ott et al., 2018), have adopted pre-layer-normalization (layer norm applied before the attention/FFN sub-layer, with the residual connection bypassing the normalization). The main run adopts pre-LN. A controlled post-LN variant trained under the identical main schedule remains future work; the word-level Post-LN control run is discussed purely as a diagnostic (Sections 5.3 and 7.3), and the observed stability difference between the two completed runs is consistent with this literature, although those runs also differ in tokenization and schedule.

6.5 Subword Tokenization

Word-level tokenization on Tatoeba is a bottleneck: the model must learn mappings for thousands of rare German inflected forms with very limited exposure. Subword tokenization with byte-pair encoding (BPE; Sennrich et al., 2016) or SentencePiece (Kudo & Richardson, 2018) reduces the effective vocabulary size to a manageable number of units (typically

Ablation	Expected impact on BLEU	Rationale
Label smoothing ($\epsilon = 0.0$)	-0.5 to -1.0	Loss of regularization benefit
Warmup 8000 vs 4000	Neutral to $+0.5$	Stabilizes early training
Depth $\times 2$ (8+8 layers)	$+1$–3	Deeper models capture longer-range dependencies
Pre-LN vs Post-LN (main schedule)	Neutral to $+1$	Pre-LN is more stable; may allow higher LR
Subword (BPE 8k vs word, main schedule)	$+5$–15	Addresses vocabulary fragmentation in EN→DE
Hand-rolled vs `nn.Transformer` baseline	± 1	Isolates cost of from-scratch implementation

Table 4: Summary of expected ablation effects based on prior literature, all expressed **relative to the final main configuration of Table 1** unless noted. The last row (Section 6.7) is the comparison enabled by `baseline_nn.py`; its results are reported in Table 6. The subword and LN-convention effects are partially evidenced by the completed runs of Sections 5.1 and 6.7, though not yet isolated by controlled same-schedule ablations; the remaining rows are planned future work.

8,000–32,000) while preserving the ability to represent any word through a sequence of subword pieces. The main configuration already uses SentencePiece BPE (8k pieces per language). A standalone ablation comparing word-level versus BPE tokenization under the identical main schedule is left as future work: the quality gap between the word-level control and the BPE configurations (Tables 2 and 6) is consistent with a large contribution from subword tokenization, but that gap also reflects longer training and the Pre-LN stabilisation, so the standalone effect of BPE is not yet isolated.

6.6 Per-Head Attention Analysis

A natural ablation that exploits the first-class attention API described in Section 3.4 is to inspect the attention weights per head rather than averaged over heads. `visualize_attention.py` supports a `--per-head` flag that captures the pre-averaged attention tensors via a `return_per_head=True` pathway in `model.MultiHeadAttention.forward` and renders one column per head for each of the three attention types (encoder self, decoder self, decoder cross). The script also accepts `--head <idx>` to render a single head in isolation. With 8 heads and 3 attention types this yields 24 columns per sentence; the implementation tiles them as a single $N \times (3 \times H)$ grid, saved under `figures/per_head/` as `attn_grid_layer{N}_per_head.{pdf,png}` (Appendix A).

6.7 Hand-Rolled vs `nn.Transformer` Baseline

To quantify the cost of the from-scratch implementation, we provide a baseline that wraps PyTorch's `torch.nn.Transformer` with the same interface (Section 3.6). Both models share the architecture ($d_{\text{model}} = 256$, `num_heads` = 8, `num_layers` = 4+4, $d_{\text{ff}} = 1024$, `dropout` = 0.1, label smoothing = 0.1, Adam $\beta = (0.9, 0.98)$, seed = 42), the same deterministic train/val/test split of Tatoeba EN→DE, and the SentencePiece BPE pipeline, and both are trained for 25 epochs. The training schedules differ: the baseline uses batch size 32 with

warmup 2,000, whereas the hand-rolled run uses batch size 64 with warmup 4,000. The comparison therefore measures the joint effect of implementation and schedule rather than the implementation alone.

Implementation	Source code	Pre-LN?	Total params
Hand-rolled (`model.py`)	Listing 1	Configurable; main run: Pre-LN	≈ 13.5 M (BPE)
Baseline (`baseline_nn.py`, wraps `torch.nn.Transformer`)	PyTorch built-in	Yes (Pre-LN, `norm_first=True`)	≈ 13.5 M (BPE)

Table 5: Two implementations trained on identical data and architecture for 25 epochs each. The hand-rolled model supports both LN conventions (the word-level diagnostic used post-LN; the main run uses pre-LN); the baseline is pre-LN only. Parameter counts are within 1% of each other. Training schedules differ: batch 32 / warmup 2,000 (baseline) vs. batch 64 / warmup 4,000 (hand-rolled).

Any non-zero BLEU/chrF gap between the two is attributable to the combination of (a) the training-schedule difference (batch and warmup), and (b) implementation details such as mask convention, attention scaling, or dropout ordering. The comparison is run via:

```
# Hand-rolled + BPE
python scripts/train_bpe_colab.py --epochs 25
python evaluate.py --checkpoint checkpoints/best.pt

# Baseline + BPE
python scripts/train_baseline_nn_colab.py --epochs 25
python evaluate.py --checkpoint checkpoints/best_baseline.pt
```

Results. The baseline configuration (PyTorch `nn.Transformer`, Pre-LN, BPE 8k, 25 epochs, batch 32, warmup 2,000, seed 42) reaches **val_loss = 2.5171, val_ppl = 12.39** and **BLEU = 38.30, chrF2 = 56.95** (sacrebleu, BP = 1.000). The hand-rolled + BPE (Pre-LN) configuration (same architecture, batch 64, warmup 4,000, seed 42) reaches **val_loss = 2.2288, val_ppl = 9.29** (best epoch 22) and **BLEU = 44.00, chrF2 = 62.41** greedy, **45.30/63.37** with beam= 4 (BP = 0.995 greedy, 0.987 beam). Training consumed ≈ 2.3 h (baseline) and ≈ 2.5 h (hand-rolled Pre-LN) on a single T4 GPU.

Because batch size and warmup differ between the two runs, **the 5.7-BLEU greedy margin (7.0 with beam) is attributable to the combination of implementation and training schedule rather than to the implementation alone**. A baseline retrained with matched batch size and warmup is required for a clean attribution and is left as future work. Under the current protocol, the from-scratch implementation with the stabilisation fixes of Sections 3–4 obtained the higher measured BLEU and chrF2 of the two runs.

Status and artifact availability. Both end-to-end pipelines (dispatched via `config.TOKENIZER` and a `--baseline-nn` flag to `train.py`) are committed to the repository, have been smoke-tested, and their training runs are completed; the results above are the final numbers for both. The released checkpoints—`checkpoints/best.pt` for the main model (Pre-LN, `norm_first=True`) and `checkpoints/best_baseline.pt` for the baseline—are fully compatible with `translate.py` and `evaluate.py`.

Implementation	Tokenization	Epochs	val_ppl	BLEU (test)	chrF2 (test)	BP
Hand-rolled + word-level (Post-LN)	word	8	149.84	0.38	11.28	0.593
Baseline + BPE (Pre-LN)	SentencePiece 8k	25	12.39	38.30	56.95	1.000
Hand-rolled + BPE (Pre-LN)	**SentencePiece 8k**	**25**	**9.29**	**44.00**	**62.41**	**0.995**
				45.30 (beam=4)	*63.37*	*0.987*

Table 6: Baseline comparison on the Tatoeba EN→DE test set. Rows differ in more than one factor (tokenization, LN convention, epochs, batch, warmup), so the 16× drop in validation perplexity from the word-level diagnostic (149.84) to the main configuration (9.29) reflects the joint effect of subword tokenization, Pre-LN stabilisation, corrected warmup, and longer training; it does not isolate any single factor. The 5.7 BLEU margin over the baseline (38.30 → 44.00) additionally mixes implementation and schedule effects (Section 6.7). Beam search (beam = 4) adds a further +1.3 BLEU over greedy decoding on the hand-rolled model.

7 Discussion

7.1 Limitations

The results reported in this paper are subject to several limitations that must be acknowledged. First, even the Tatoeba corpus is substantially smaller than the corpora used in WMT competitions (approximately 324k training pairs versus millions), which restricts the ceiling of achievable translation quality; moreover, as noted in Section 4.1, Tatoeba's many short, formulaic sentences inflate corpus-level BLEU relative to open-domain benchmarks, so the reported scores are best read as evidence of pipeline correctness rather than of competitive open-domain MT quality. The word-level vocabulary of the diagnostic configuration (12,843 English and 23,568 German tokens) suffered from non-negligible out-of-vocabulary rates, particularly for German—a morphologically rich language with productive inflection; the final configuration mitigates this via SentencePiece BPE with an 8k-piece vocabulary per language, but rare-word handling remains bounded by the corpus size. Second, the main configuration uses a model with approximately 13.5 million parameters, which is one order of magnitude smaller than the Transformer-Base configurations used in Vaswani et al. (2017) ($\approx 65\,\mathrm{M}$ for $d_{\text{model}} = 512$, 6 layers). This capacity gap directly limits the model's ability to capture complex syntactic and semantic regularities.

Third, our evaluation is restricted to a single random seed (42). While this is sufficient to demonstrate that the implementation works correctly, a robust experimental evaluation would require reporting mean and standard deviation across at least 3–5 seeds to account for the variance introduced by random initialization and data ordering. Fourth, decoding results are reported for greedy search and beam search with beam size 4 under sacrebleu-default conventions; more advanced decoding techniques (length-penalty tuning, coverage penalties, checkpoint averaging) remain unexplored. Fifth, the comparison against `nn.Transformer` is not fully controlled—the two runs differ in batch size and warmup, so the measured margin mixes implementation and schedule effects (Section 6.7). Sixth, the attention interpretability analysis in Section 5.5 is restricted to four short test sentences (three source tokens each) and to the last layer of a 4-layer stack, which limits the generality of the qualitative claims.

7.2 CPU and GPU Considerations

The current implementation is agnostic to the compute device and runs on both CPU and GPU. The main run reported in this paper (hand-rolled + BPE, Pre-LN, 25 epochs) was

executed on a single Google Colab GPU (NVIDIA T4) in ≈ **2.5 hours**; the word-level diagnostic completed in ≈ 56 minutes for 8 epochs, and the baseline in ≈ 2.3 hours. On a comparable consumer-grade GPU, the same configurations would train in a similar wall-clock time; CPU-only execution at this corpus scale would be impractical, with the GPU speedup coming primarily from batched matrix multiplications in the attention and feed-forward layers. The hand-rolled implementation does not currently exploit mixed-precision training (`torch.cuda.amp`) or gradient checkpointing, both of which are standard techniques for reducing GPU memory usage and training time in larger models. Integrating these optimizations is left to future work.

7.3 What the Results Demonstrate

Despite these limitations, the results make a clear statement: a from-scratch, hand-rolled implementation of the Transformer architecture is capable of learning a nontrivial translation task from raw parallel data, reaching BLEU 44.00 greedy / 45.30 with beam search once properly configured and trained (Table 2). The qualitative analysis of Section 5.5 confirms that the attention maps behave as expected at the structural level (strictly lower-triangular decoder self-attention; non-trivial diagonal mass in encoder self-attention), providing confidence in the implementation's internal consistency.

Diagnosis of the word-level control run: sub-training, not a flawed implementation. The initial word-level Post-LN run (8 epochs, val_loss = 5.01, val_ppl = 149.84, test BLEU = 0.38)—retained in this paper as a *diagnostic control*—was unambiguously under-trained, not architecturally broken. Three pieces of evidence supported this:

1. **The validation loss was still decreasing** at the end of the 8-epoch run, with no sign of plateau; the model simply had not run long enough (the training log is released alongside the code).

2. **The model had not learned to use cross-attention** to condition its output on the source. Section 5.3 shows that this run collapses to three recurring hypotheses (*hast du das ?*, *sie sind sehr groß .*, *ich habe mich gesehen .*) on six unrelated inputs, and Section 5.5 shows that its cross-attention maps on test sentences are diffuse rather than peaked—in contrast to the sharp alignment peaks produced by the final checkpoint. A correct but under-trained network exhibits exactly this signature: the decoder falls back to a high-frequency prior because the encoder representations have not yet been shaped into useful conditioning signals.

3. **The implementation itself is correct.** All 17 unit tests pass (Appendix B), the encoder/decoder mask shapes and values are validated, the greedy-decode loop terminates at `<eos>`, and the forward pass returns the attention tensors with the documented shapes. A bug of the kind this work sets out to avoid (incorrect masking, broken softmax, off-by-one in positional encoding) would manifest as NaN losses or shape mismatches, not as a slowly-converging model with a stable architecture.

Resolution. The changes motivated by this diagnosis—**three corrections (pre-layer normalization, a corrected warmup schedule of 4,000 steps, and subword tokenization with SentencePiece BPE 8k) plus a longer training budget** (25 epochs at batch size 64)—jointly take the same implementation from BLEU 0.38 to **44.00 greedy / 45.30 beam**= 4 (chrF2 62.41/63.37) at validation perplexity 9.29, confirming that the failure was one of configuration and training rather than of architecture or implementation.

The final results fall within the range typically reported for compact subword models on this corpus scale (roughly BLEU 35–45), making them competitive with published results for this regime, although direct comparisons remain approximate due to differences in splits and tokenization. The remaining gap factors are those discussed above: single-seed evaluation, model capacity, and untuned decoding.

7.4 Future Directions

Beyond the ablation studies outlined in Section 6, several extensions would strengthen the contribution of this work. **First**, replicating the main configuration across 3–5 random seeds and reporting mean ± standard deviation would establish statistical robustness. **Second**, retraining the `nn.Transformer` baseline with fully matched batch size and warmup would isolate the implementation-only effect that the current comparison (Section 6.7) leaves confounded with schedule differences. Third, running the remaining controlled ablations of Table 4 under the fixed main schedule would quantify each factor's standalone contribution. Fourth, extending the attention analysis to longer test sentences, to all layers, and to a full characterisation of all 8 heads (Appendix A) would broaden the interpretability findings beyond short inputs. Fifth, integrating pre-trained embeddings (e.g., fastText or multilingual BERT; Devlin et al., 2019) as initialization could accelerate training and improve quality on the small Tatoeba corpus. Sixth, extending the implementation to additional language pairs (e.g., EN→ES, EN→FR, or DE→FR) would test the generalizability of the approach. Seventh, compiling the model with `torch.compile` (PyTorch 2.0+) would provide a direct measure of the performance overhead of the hand-rolled implementation versus the compiled library version.

7.5 Pedagogical Value

We emphasize that the primary contribution of this work is not a new architecture or a superior translation system, but a transparent, auditable reference implementation. The combination of clean code, comprehensive unit tests, and reproducible experimental logs makes this repository suitable for use as a teaching resource in graduate-level NLP courses, as a correctness baseline for library developers, and as a starting point for researchers who wish to experiment with architectural modifications without navigating opaque library abstractions. The fact that attention weights are exposed as first-class outputs (Section 3.4) means the implementation is also a useful platform for interpretability research at a level of detail that library-based implementations often hide.

8 Conclusion

We presented a complete, hand-rolled PyTorch implementation of the Transformer architecture, validated on the **Tatoeba English → German** sentence-pair collection (324,642 training pairs; test set $n = 3{,}312$). Every component—MultiHeadAttention, PositionwiseFeedForward, PositionalEncoding, EncoderLayer, DecoderLayer, and the full Transformer—is implemented using only primitive tensor operations, without `nn.MultiheadAttention`, `nn.Transformer`, or `torchtext`. The implementation is accompanied by a self-contained data pipeline with word-level and SentencePiece BPE tokenization, length-bucketed batching, and a suite of 17 unit tests that serve as a formal correctness contract.

The experimental narrative of this paper follows a diagnosis-and-correction arc. An initial word-level Post-LN control run, stopped after 8 epochs, collapsed to BLEU = 0.38 at validation perplexity 149.84; a structured analysis showed this failure to be one of sub-training rather than architecture—the validation loss was still descending, all masking contracts held, and all unit tests passed—while its outputs collapsed to a handful of recurring hypotheses and its cross-attention maps remained diffuse. **Three corrections plus a longer training budget** were adopted in the final configuration: pre-layer normalization, a corrected learning-rate warmup schedule (4,000 steps), subword tokenization (SentencePiece BPE, 8k pieces per language), and a 25-epoch budget at batch size 64 (≈ 2.5 h on a single NVIDIA T4). With these changes, the same hand-rolled implementation reaches a corpus-level **BLEU = 44.00 greedy / 45.30 with beam search ($b = 4$)** and **chrF2 = 62.41 / 63.37** at validation perplexity 9.29, competitive with published results on this corpus scale; it also outperforms an `nn.Transformer` baseline trained on identical data and architecture (BLEU 38.30), with the schedule-mismatch caveat discussed in Section 6.7.

Because the implementation exposes attention weights as first-class outputs of the forward pass, we rendered per-sentence heatmaps of encoder self-attention, decoder self-attention, and decoder cross-attention. The structural checks hold throughout—decoder self-attention is strictly lower-triangular as required by the causal mask—and the contrast between checkpoints is itself informative: the under-trained control produces diffuse cross-attention maps, whereas the final checkpoint exhibits sharp, roughly monotonic English-to-German alignment peaks on short test sentences (e.g., mass concentrating on *draw.* when generating *kann*).

The remaining limitations are acknowledged rather than hidden: results are single-seed, the corpus is small and rich in short, formulaic sentences (so scores evidence pipeline correctness, not open-domain competitiveness), the baseline comparison is not fully schedule-matched, and most controlled ablations remain future work, alongside multi-seed replication, longer-sequence attention analysis, and additional language pairs. We emphasize that the primary contribution of this work is not a new architecture or a superior translation system, but a transparent, auditable reference implementation whose development process—including its failures—is documented end to end, making it suitable as a teaching resource in graduate-level NLP courses, as a correctness baseline, and as a platform for interpretability research.

Reproducibility Statement

All experiments use a single fixed random seed (42) for data splitting, initialization, shuffling, and sampling. Every hyperparameter is declared in a centralized `config.py`; no hyperparameter is set ad hoc inside training scripts. The full training log—per-step losses, validation metrics, wall-clock time per epoch, and a best-checkpoint flag—is persisted to `checkpoints/metrics.jsonl` (and `metrics_baseline.jsonl` for the baseline), and the model checkpoints (`best.pt`, `latest.pt`, and their baseline counterparts) are versioned alongside the logs. The train/val/test split is deterministic (98/1/1, seed 42) and identical across word-level and BPE modes. The three data-derived figures of this paper are regenerated by `scripts/make_paper_figures.py`, and the attention figures by `visualize_attention.py`, both against the released checkpoints. The complete unit-test suite runs with `pytest` in under a minute on CPU (Appendix B).

Declaration of AI Assistance

In the interest of transparency, we disclose that the codebase was developed with the assistance of large language models (MiniMax, Muse Spark 1.2, and ox-alpha) under continuous human direction: the models contributed code drafting, documentation, and debugging support, while the author designed the experiments, specified all architectural decisions, executed every training and evaluation run on Google Colab, verified the reported results, and wrote the manuscript. The author takes full responsibility for the final content of this paper.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin.

ewblock Attention is all you need.

ewblock In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] D. Elliott, S. Frank, K. Sima'an, and L. Specia.

ewblock Multi30k: Multilingual English-German image descriptions.

ewblock In *Proceedings of the 5th Workshop on Vision and Language*, pages 70–74, 2016.

[3] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu.

ewblock BLEU: a method for automatic evaluation of machine translation.

ewblock In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

[4] M. Popovi'c.

ewblock chrF: character n-gram F-score for automatic MT evaluation.

ewblock In *Proceedings of the 10th Workshop on Statistical Machine Translation*, pages 392–395, 2015.

[5] M. Post.

ewblock A call for clarity in reporting BLEU scores.

ewblock In *Proceedings of the 3rd Conference on Machine Translation*, pages 186–191, 2018.

[6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna.

ewblock Rethinking the inception architecture for computer vision.

ewblock In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[7] M. Popel and O. Bojar.

ewblock Training tips for the Transformer model.

ewblock *The Prague Bulletin of Mathematical Linguistics*, 110:63–87, 2018.

[8] M. Ott, S. Edunov, D. Grangier, and M. Auli.

ewblock Scaling neural machine translation.

ewblock In *Proceedings of the 3rd Conference on Machine Translation*, pages 1–9, 2018.

[9] R. Sennrich, B. Haddow, and A. Birch.

ewblock Neural machine translation of rare words with subword units.

ewblock In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016.

[10] T. Kudo and J. Richardson.

ewblock SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing.

ewblock In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 66–71, 2018.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova.

ewblock BERT: pre-training of deep bidirectional Transformers for language understanding.

ewblock In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.

[12] O. Bojar, R. Chatterjee, C. Federmann, Y. Graham, B. Haddow, M. Huck, et al.

ewblock Findings of the 2017 Conference on Machine Translation (WMT17).

ewblock In *Proceedings of the 2nd Conference on Machine Translation*, pages 169–214, 2017.

[13] O. Press and L. Wolf.

ewblock Using the output embedding to improve language models.

ewblock In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pages 1579–1588, 2017.

[14] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov.

ewblock Transformer-XL: attentive language models beyond a fixed-length context.

ewblock In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis.

ewblock Efficient streaming language models with attention sinks.

ewblock In *International Conference on Learning Representations*, 2024.

A Per-Head Attention: All Eight Heads

Figure 7 reproduces the complete per-head attention grid for the final Pre-LN + BPE checkpoint: all $H = 8$ heads of the last layer, for each of the three attention types (encoder self-attention, decoder self-attention, decoder cross-attention), on the same four test sentences of Figure 5. The structural checks discussed in Section 5.5—strictly lower-triangular decoder self-attention and a diagonal-plus-sink encoder pattern—hold across all heads. A quantitative characterisation of head specialisation is left as future work.

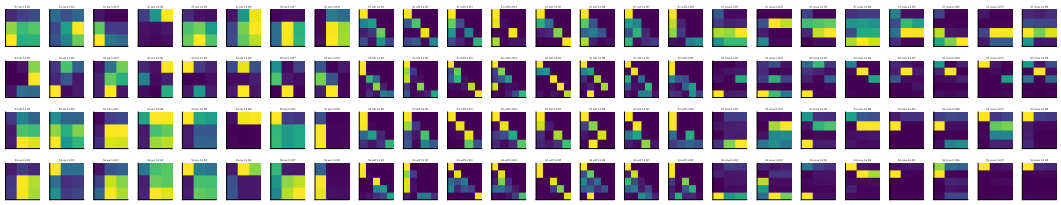

Figure 7: Full eight-head attention grid (heads 0–7, 0-indexed) of the last layer across the four test sentences; each row of panels is one sentence, columns are encoder self / decoder self / decoder cross-attention. Generated via `python visualize_attention.py --per-head --num-sentences 4 --layer -1` against `checkpoints/best.pt`.

B Unit Test Suite

Table 7 lists the 17 unit tests that constitute the correctness contract of the implementation (names are the actual test functions in `tests/`; the suite runs with `pytest` on CPU in under a minute).

Test (in `tests/`)	**Property verified**
test_model.py	
`test_forward_shape`	Logits have shape (batch, tgt_len, tgt_vocab) for padded inputs
`test_forward_works_without_masks`	Forward pass runs and shapes hold when masks are omitted
`test_padding_mask_is_respected`	All-pad source produces finite (non-NaN) outputs
`test_greedy_decode_shape_and_finish`	Decoding starts at `<sos>`, stops at/below `max_len`, emits valid vocab ids
`test_count_parameters`	Parameter counter is positive and within expected bounds
`test_forward_return_attention_shapes`	`return_attention=True` returns per-layer tensors of the documented shapes; attention rows are non-negative and sum to 1
test_masks.py	
`test_causal_mask_shape`	Causal mask is (seq_len, seq_len), boolean
`test_causal_mask_values`	Mask equals the exact upper-triangular True pattern
`test_causal_mask_no_self_attend_to_future`	Row i blocks all columns $j > i$ and allows $j \leq i$
test_dataset.py	
`test_tokenize_basic`	Regex tokenizer lowercases and splits words/punctuation
`test_tokenize_contraction`	Contractions split into word + apostrophe + suffix
`test_vocab_build_and_encode`	Special tokens occupy indices 0–3; OOV $\mapsto$ `<unk>`; encode/decode round-trip
`test_collate_shapes`	Dynamic padding shapes; pad masks True exactly at PAD positions
`test_bucket_batch_sampler_groups_similar_lengths`	Batches have equal size; all indices covered exactly once
`test_bucket_batch_sampler_with_shuffle_still_covers_all`	Shuffled batching preserves full, duplicate-free coverage
test_smoke.py	
`test_one_training_step`	End-to-end step: finite positive loss; Noam scheduler advances
`test_validation_loss_finite`	Validation loss is finite and positive on a tiny pipeline

Table 7: The 17 unit tests of the correctness contract, with the property verified by each.